

SHOPLINE

Grafana 全家桶的最後一塊拼圖

自建 Sentry 帶你攻略 Error Tracking

Jeff Lin

Cloud Engineer

SHOPLINE

關於我

Jeff Lin

Cloud Engineer @ SHOPLINE

7 年工程經驗

2 年網路維運 + 5 年軟體開發維運

Cloud Infrastructure (IaC) , CI/CD,
Monitoring

- 目前負責 SHOPLINE 的服務部署與監控工具維運，今天分享的 Sentry 自建，就是其中一環。



LinkedIn

我們的可觀測性現狀

三大支柱，看似完備



Prometheus

Metrics 指標監控



Loki

Log 日誌查詢



Tempo

Traces 鏈路追蹤



看似完美的監控堡壘，當半夜 Oncall 響起時，我們真的能秒解問題嗎？

事故發生時的真實場景



📄 **Dev 的崩潰：**「只有這一行 Error 訊息，我怎麼知道前因後果？呼叫參數是什麼？上下文在哪裡？」

Log $\neq$ Error Tracking

兩者設計目的不同，混用會讓問題更難解。

Logs (Loki)

- 線性時間軸，依序輸出
- 非結構化或半結構化文字
- 數量龐大，充滿雜訊
- 缺乏聚合與分組能力

Error Tracking (Sentry)

- 自動聚合相同錯誤 (Grouping)
- 完整上下文 (Context)
- Stack Trace + Local Variables
- Issue 生命週期管理

Sentry 帶來的核心價值

從「大海撈針」到「精準定位」



自動聚合

相同根因的錯誤自動歸類，一眼看清影響範圍與發生頻率。



完整堆疊追蹤

Stack Trace 直指問題程式碼行，附帶當下的 Local Variables。



縮短 MTTR

Dev 不再靠猜測，直接進入修復流程，平均修復時間顯著下降。

Sentry UI 實際操作 (1/3) : Project Overview

S Sentry
jeff.lin@shopline.co...

Issues

Projects

Explore

Replays

Discover

Insights

User Feedback

Alerts

Dashboards

Releases

Stats

Settings

Help

What's new

Collapse

Projects > Project Details

cloud-permissions-api

View All Issues Create Alert

All Envs Mar 18, 11:23 AM-Mar 20, 3:50 PM Search by release version, build, package, or stage

Crash Free Sessions
Start Setup

Crash Free Users
Start Setup

Number of Releases
2

Apdex
1

Number of Errors

10

8

6

4

2

0

Mar 18 12:00 PM

Mar 19 12:00 AM

Mar 19 12:00 PM

Mar 20 12:00 AM

Mar 20 12:00 PM

Total Errors 10 Display: Number of Errors

Failure Rate

100%

80%

60%

40%

20%

0%

Mar 18 12:00 PM

Mar 19 12:00 AM

Mar 19 12:00 PM

Mar 20 12:00 AM

Mar 20 12:00 PM

Total Transactions 36,370 Display: Failure Rate

All Issues 1 New Issues 0 Unhandled 0 Regressed 0 Resolved 0

Open in Issues Open in Discover

ISSUE	LAST SEEN	AGE	EVENTS	USERS	ASSIGNEE
*errors.errorString test error from K6 CLOUD-PERMISSIONS-API-1 cloud-permissions-api/routers/api/v1 in func1	2wk ago	4wk	10	0	

Sentry UI 實際操作 (2/3) : Issue Detail

S Sentry

jeff.lin@shopline.co...

Issues

Projects

Explore

Replays

Discover

Insights

User Feedback

Alerts

Dashboards

Releases

Stats

Settings

Help

What's new

Collapse

Issues > CLOUD-PERMISSIONS-API-1

Try New UI

All Envs

Open in Discover

Archive

Resolve

EVENTS 104k

USERS 0

PRIORITY

*errors.errorString Ongoing

test error from K6

Details Activity User Feedback Attachments Tags All Events Merged Issues Similar Issues Replays

Event ID: a5c410e7 Apr 1, 7:02 PM

Event Highlights

handled --

level error

url

Trace: Trace ID

View Full Trace

Stack Trace

*errors.errorString

test error from K6

/cloud-permissions-api/routers/api/v1/sentry.go in func1 at line 47

Set up Code Mapping In App

/cloud-permissions-api/routers/api/v1/sentry.go in SendSentryError at line 39

In App

Called from: /go/pkg/mod/github.com/gin-gonic/gin@v1.10.0/context.go in (*Context).Next

1 Show 8 more frames

POST /sentry/test

Formatted curl

Body

{
 "level": "error",
 "message": "test error from K6"
}

Assigned To No one

Last 24 Hours 0

Last 30 Days 104k

Last Seen 13 days ago in release 84dbc1a10892

First Seen a month ago in release 489680f3fef1

Issue Tracking

Bitbucket Issue

Jira Issue

All Tags

url 100%

Sentry UI 實際操作 (3/3) : Issue Detail

S Sentry
jeff.lin@shopline.co...

Issues

Projects

Explore

Replays

Discover

Insights

User Feedback

Alerts

Dashboards

Releases

Stats

Settings

Help

What's new

Collapse

Headers

Content-Length48

Content-Typeapplication/json

Host

User-AgentGrafana k6/1.1.0

Show More

Tags

endpoint/sentry/test

environmentpreview

levelerror

os

namelinux

release84dbc1a10892

runtimego go1.24.13

namego

server_namecloud-permissions-api-fcc9fbf98-8hnsn

sourceapi-test

url

Contexts

Runtime

go_maxprocs8

go_numcgocalls0

go_numroutines16

Namego

Versiongo1.24.13

Device

Architectureamd64

num_cpu8

Trace Details

Span ID14868025df967b8e

Statusunknown

Trace ID

Operating System

Namelinux

Additional Data

request_methodPOST

FormattedRaw

100%

release84dbc1a10892

99%

browsercurl 8.6.0

80%

browser.namecurl

100%

endpoint/sentry/test

100%

environmentpreview

100%

levelerror

100%

os.namelinux

100%

runtimego go1.24.13

100%

runtime.namego

100%

server_namecloud-permissions-api-f...

7%

sourceapi-test

100%

Viewers (2)

R

H

為何選擇自建 Self-Hosted ?

SaaS 版本很香，我們卻有其他考慮

→ 資料隱私合規

敏感的錯誤上下文（含用戶資料），希望自主掌控。

→ 預算控制

SaaS 事件量費用難以預測，事件量大時，自建成本可控制。

→ 初期流量評估

自建環境可依實際流量動態調整規格，避免過早承擔高額授權費。

Sentry Self-Hosted 組件解析

光是組件數量，就足以讓人頭皮發麻。這也是為什麼部署比想像中複雜。

● 入口層

Relay

Web

● 訊息佇列

Kafka

RabbitMQ

Redis

● 工作處理

Celery Workers

Ingest Consumer

Snuba Consumer

Subscription

Consumer
Symbolicator

Post-Process FWD

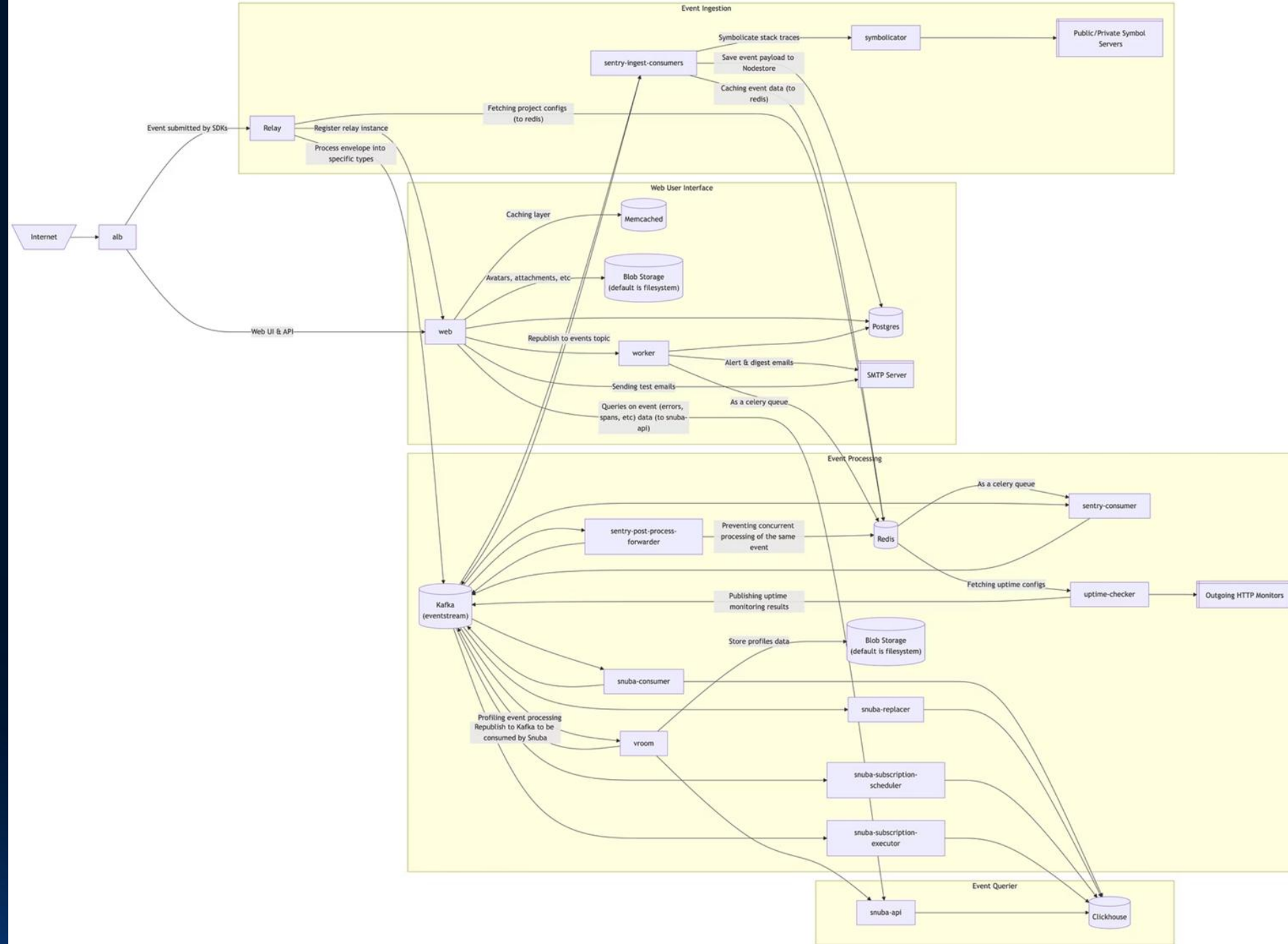
● 儲存層

PostgreSQL

Redis

ClickHouse

❏ 組件多只是開始，資料流才是真正的複雜。



從 SDK 送出事件到 UI 顯示 Issue，中間經過 6+ 個服務、2 套訊息佇列、3 個儲存系統。任何一個環節出問題，都可能導致事件丟失或延遲。

踩雷 1：部署踩坑

部署 Sentry 要多久？

 幾天，helm install 嘛

 頂多一週吧？

 目標很單純：用 Helm Chart 把 Sentry 部署起來，讓它能接收事件、顯示 Issue。看起來就是 helm install 一行指令的事。

踩雷 1：部署踩坑

沒想到...helm install 之後的驚喜

Chart values 繁雜

官方 Helm Chart 有數百個可設定項，預設值針對 demo 環境調校，直接套用到生產環境會資源不足。

Pod 啟動順序

Sentry 各服務有特定的啟動相依性，readiness probe 設定不當會導致服務反覆重啟。

初始化腳本

各組件有個字的初始化腳本，並也是有相依性，且腳本跑完後 pod 就會消失

⚠ 我們以為 helm install 就結束了，結果只是噩夢的開始。

踩雷 1：部署踩坑

還好...讓部署不再是賭博



✓ 從 self-hosted repo 反推 helm 設定

官方 self-hosted repo 是最真實的參考，把它的环境變數和資源限制對照翻譯成 Helm values，比直接看 chart repo 更可靠。



✓ 先在 staging 驗證，再上 production

用最小可用規格在 staging 跑通完整流程，確認事件能從 SDK 流到 UI，再複製設定到 production 並調高資源。

⚠ 避坑建議：預留部署緩衝時間，不要在 release 前一天才開始部署 Sentry。

踩雷 2：基礎設施選型

如果是你，這些組件你會怎麼選？

降低維運複雜度

Sentry 組件眾多，希望找到最省力的維運方式。

確保高可用（HA）

生產環境的 Error Tracking 系統不能輕易掛掉，各組件都需要 HA 方案。

控制成本

Managed 有額外費用，自建可以省錢，需要在兩者之間找到平衡。

📌 我們的策略：能自建就自建，不得不 Managed 的才 Managed。聽起來很合理——直到我們發現有些組件的 HA 比想像中複雜得多。

踩雷 2：基礎設施選型

沒想到...HA 的複雜度遠超預期

 自建 Redis HA 比想像中複雜
Redis Sentinel / Cluster 模式在 EKS 上的
部署和維運相當繁瑣，Sentry Client 端連
線設定無法調整。

 Chart 內建的 ClickHouse 無 HA
Sentry 官方 Helm Chart 內建的
ClickHouse 不支援 HA 配置，單點故障風
險高。

 我們一開始低估了「自建」這兩個字背後的維運成本。

踩雷 2：基礎設施選型



✓ PostgreSQL → Managed
公司既有技術選型，有 DBA
協助維運，無懸念直接沿用



✓ Kafka → 維持自建
團隊成員有維運經驗，
Managed Kafka 費用偏高。



✓ Redis → 遷移至 Managed
HA 需改 client 連線方式，遷
移至 Managed 服務以避免單
點故障。



✓ RabbitMQ → 整合至 Redis
減少一個需要維護的組件。
Celery Queue 可以由 Redis
承擔，降低維運負擔。



✓ ClickHouse → 維持自建
Chart 內建的 ClickHouse 不
支援 HA，改用 Altinity
ClickHouse Operator 支援
HA。

⚠ **避坑建議：**先列出每個組件有無 HA 需求，評估自建維運成本再決定選型。事後遷移的代價遠高於事前評估。

踩雷 3：SDK 配置陷阱

接 SDK 嘛，能有多難？

收集應用程式錯誤事件

在各語言服務接入 Sentry SDK，讓錯誤自動上報，不再靠人工撈 Log。

追蹤使用者行為

啟用 PII 收集，取得使用者 IP，方便追溯錯誤發生時的使用者情境。

控制資料量

只收 Error 事件，不收 Transaction，避免不必要的儲存與處理負擔。

📌 看起來只是裝個 SDK、改幾行設定，應該很快就能搞定。

踩雷 3：SDK 配置陷阱

沒想到...配置範例咬了我們一口

 配置範例把 `trace_sample_rate` 設為 1.0
Sentry 建立 Project 時提供的配置範例，預設將 `trace_sample_rate` 設為 1.0。照著範例使用後，在不知情的情況下收集了 Transaction 資料，導致資料量暴增。

 `send_default_pii=true` 多收了一些東西
為了收集使用者 IP 而啟用 `send_default_pii=true`，結果瀏覽器 Cookies 也被送進 Sentry。出於隱私合規考量，這些 Cookies 不應該被收集。

 兩個坑的共同點：範例程式碼預設開啟，但副作用沒有明確警告。

踩雷 3：SDK 配置陷阱

證據：官方串接範例預設

Configure SDK

If you have the `flask` package in your dependencies, the Flask integration will be enabled automatically when you initialize the Sentry SDK. Initialize the Sentry SDK before your app has been initialized:

```
import sentry_sdk
from flask import Flask

sentry_sdk.init(
    dsn="http://5cae33aff08ff5449820542170d01485@sentry-relay-preview.internal.shopline/17",
    # Add data like request headers and IP for users,
    # see https://docs.sentry.io/platforms/python/data-management/data-collected/ for more info
    send_default_pii=True,
    # Set traces_sample_rate to 1.0 to capture 100%
    # of transactions for tracing.
    traces_sample_rate=1.0,
)

app = Flask(__name__)
```

踩雷 3：SDK 配置陷阱

還好...不信任預設值，每個參數都自己指定



✓ 明確寫上 `traces_sample_rate=0`

不照抄範例，每個語言的 SDK 初始化時都自己寫上 `traces_sample_rate=0`，確保不收集 Transaction 資料。



✓ 用 Sensitive Fields 過濾敏感資料

在 Sentry 設定中，將 Cookies 加入 Global Sensitive Fields，Sentry 會在接收事件時自動隱藏這些欄位。既保留 IP 收集，又避免 Cookies 外洩。

⚠ **避坑建議：**建立公司內部 SDK 設定範本，不要讓工程師去猜預設值是什麼。

踩雷 4：版本地雷

用最新穩定版，應該沒問題吧？

選用最新穩定版 25.7.0

這是當時的最新穩定版，文件完整，
changelog 沒有明顯的 breaking change，
看起來是個安全的選擇。

設定 Metrics Alert 告警規則

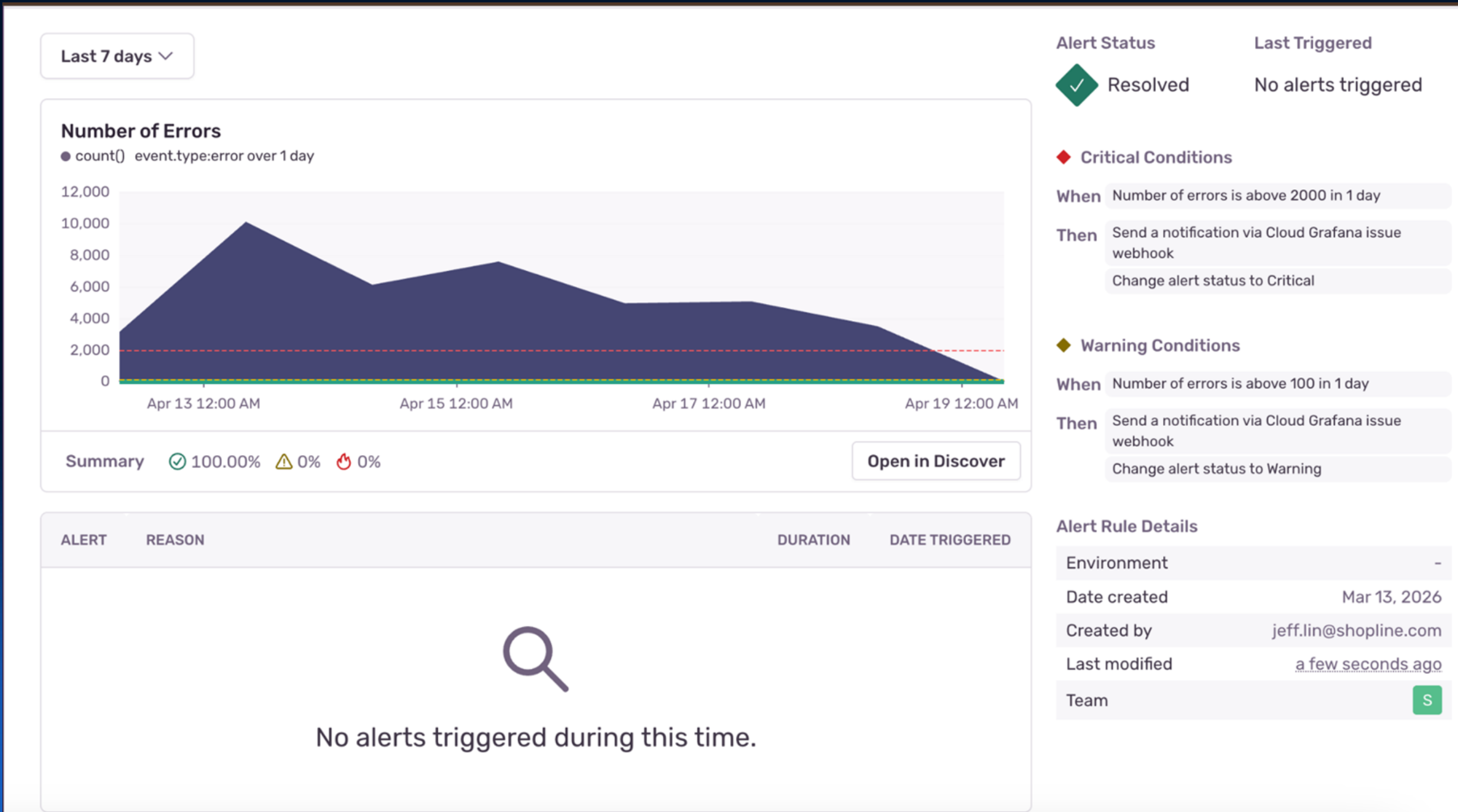
設定告警規則，例如「error 發生頻率超過每
分鐘 100 次，發送 Slack 通知」，讓 Sentry
成為主動告警的工具。

部署完成，SDK 串接成功，信心滿滿地在生產環境開始正式使用 Sentry。

❏ 一切看起來都很順利。然後，我們發現了一件讓人冷汗直流的事。

踩雷 4：版本地雷

沒想到...告警靜默，我們渾然不知



踩雷 4：版本地雷

證據：GitHub issue 回報

Alerts on Number of Errors does not work #101581

✓ Closed

#103087



yildizozgur opened on Oct 16, 2025 · edited by yildizozgur

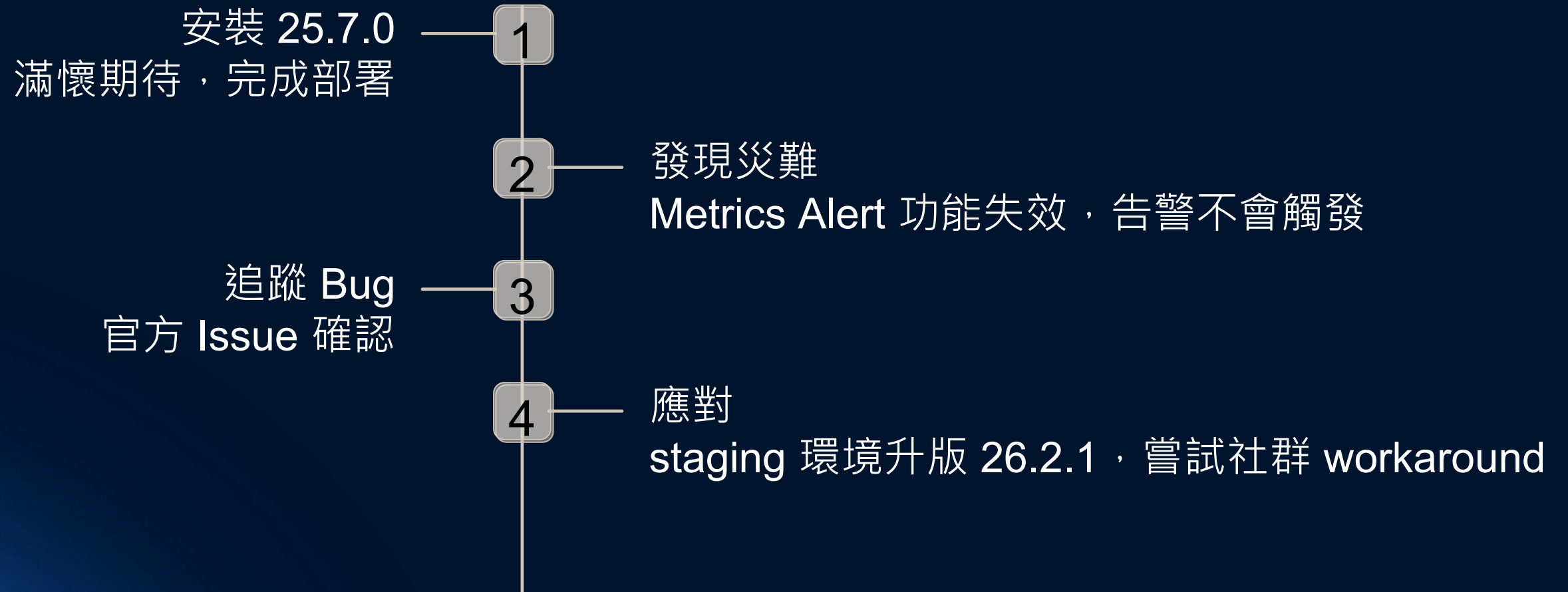
Edits ▾ ...

Hi,

Our sentry version is 25.8.0. When We try to define an alert on Number of Errors, even the conditions are met(for testing we have set very basic level) , it does not trigger any alert. What possible reason may cause this? Does anyone faced same or similar case?

Kind regards.

踩雷 4：版本地雷



❏ 教訓：升級前後，務必在驗證 Alert 端到端流程。

踩雷 4：版本地雷

還好...建立輔助監控



- ✓ Rollback 至上一個穩定版本
確認 Bug 在最新 26.2.1 尚未解決後，立即 rollback 回上一個穩定版本。



- ✓ 改用 Issue Alert 取代 Metrics Alert
Issue Alert 的彈性不如 Metrics Alert，但在我們的場景目前還算夠用。改用 Issue Alert 可以維持基本的告警能力。

⚠ 避坑建議：不要盲目追最新版。等社群回報穩定後再升級，是自建開源軟體的生存法則。

踩雷 5：資源調校

Sentry 跑起來了，然後呢？

承載真實的事件量

從 staging 切換到 production，事件量從每天幾百筆變成幾萬筆，需要確保系統能穩定處理。

事件即時出現在 UI

工程師期望錯誤發生後幾秒內就能在 Sentry 看到 Issue，而不是幾分鐘後才出現。



官方建議規格是起點，但實際需求跟事件量有關，上線前務必壓測驗證。

踩雷 5：資源調校

沒想到...系統跑起來了，但事件不見了

Celery Workers

高事件量時 celery queue 持續堆積
，worker 消費速度跟不上生產速度
。

Kafka Consumer Lag

Consumer group lag 持續累積，事
件處理延遲，影響 Sentry 即時性

❏ 除了版本地雷，我們在資源調校上也踩了很多坑。

踩雷 5：資源調校

還好...監控先行，壓測驗證



✓ Celery Workers：增加 worker 數量

先調高 replica 數驗證，後來改用 autoscaling 機制。



✓ Kafka：增加 Partition 和 Consumer
增加 partition 和 consumer 數讓事件可以平行處理，並充分利用新增的 partition。



✓ 監控 Queue Size 與 Consumer Lag

針對 RabbitMQ queue length 與 Kafka consumer lag 建立監控告警，讓問題在惡化前就被發現。

⚠ 避坑建議：先模擬真實事件量做壓測，確認 lag 能有效消化再上線生產環境。

踩雷 6：USER 管理自動化

串接個 API 嘛，能有多難？

自動新增 User

設計自動化流程，讓新成員加入時自動被加入 Sentry 組織，不需要手動操作。

自動刪除 User

成員離職或權限異動時，自動將其從 Sentry 移除，確保存取控制即時生效。

降低維運負擔

透過 API 整合 HR 系統，讓 User 生命週期管理全程自動化。

❏ 看起來只要串接 Sentry API 就能搞定，應該不複雜吧？

踩雷 6：USER 管理自動化

沒想到...Remove ≠ Delete

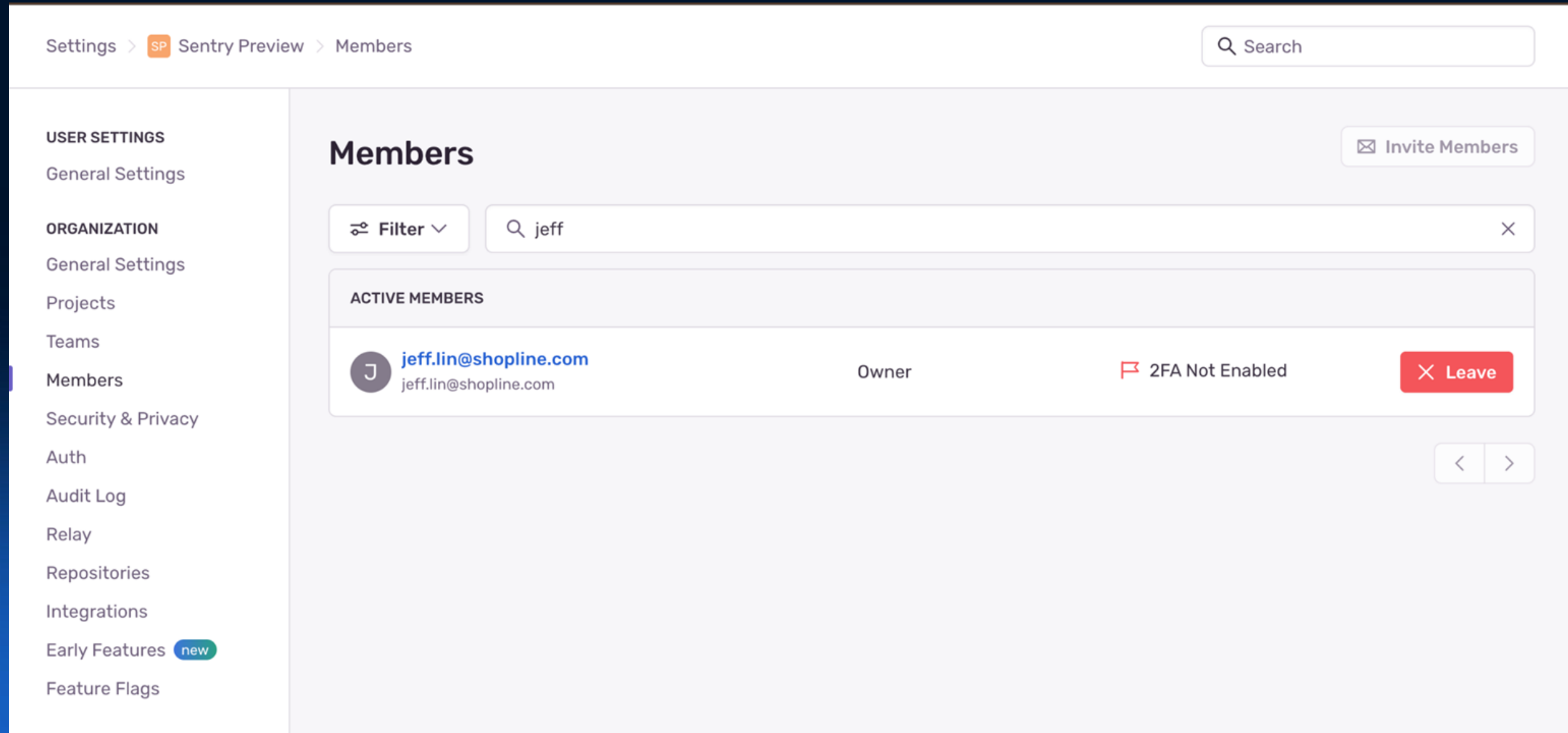
 Remove User API 其實是「Leave Org」
Sentry 的 Remove Member API 並不是刪除帳號，而是讓該 user 離開組織。帳號本身仍然存在於 Sentry 系統中。

 重建 User 報錯：User 已存在
在我們嘗試重新建立同一個 user 時，Sentry 回傳錯誤——因為該帳號從未被真正刪除，只是離開了組織。自動化流程因此卡住，無法正常重建成員。

 「Remove」和「Delete」是兩件事，但 API 文件的命名讓人很容易混淆。

踩雷 6：USER 管理自動化

證據：Member 頁面的操作是「Leave」，不是「Delete」



踩雷 6：USER 管理自動化

還好...Shell 清帳 + 事前驗證 API 行為



✓ 透過 Sentry Shell 手動刪除殘留帳號
進入 Sentry 容器執行 `sentry shell`，找出所有 Leave Org 但未真正刪除的 `user`，逐一清除乾淨，讓自動化流程得以正常重建帳號。



✓ 事前確認 API 的實際行為

在串接任何 Sentry API 之前，先用測試帳號實際驗證 API 的行為，而不是只看文件名稱就假設語意。

⚠ 避坑建議：自動化流程上線前，務必在 staging 環境跑完整的 `create` → `remove` → `recreate` 循環，確認每個步驟的實際效果。

我們到哪了？

✓ 現況

Sentry 穩定運行中

6 個坑都踩過了，系統穩定承載生產流量，Dev / Ops 都在日常使用。

Error Tracking 補上最後一塊

從 Metrics, Logs, Traces，現在加上

Error Tracking，可觀測性拼圖完整了。

→ SOON Next Action

Grafana Sentry Data Source

計畫透過 Grafana Plugin 將 Sentry Issues 拉入 Dashboard，讓 Dev / Ops 不再需要切換工具。

Sentry Cloud vs. 自建：怎麼選？

Sentry Cloud (SaaS)

優點

- 零維運負擔，開箱即用
- 自動升級，永遠用最新版
- 官方 SLA 保障，穩定性有保證
- 適合快速驗證、小團隊

缺點

- 資料託管，可能有隱私疑慮
- 事件量大時費用快速攀升
- 客製化空間有限

Self-Hosted

優點

- 資料完全自主，減少機敏資料外流風險
- 大量事件下成本可控
- 可以客製化（Relay 規則、Retention 等）

缺點

- 需要專人維運（Kafka、ClickHouse、Redis...）
- 版本升級有地雷，需建立降版機制

誰適合哪個？

☁️ 選 Sentry Cloud 如果你是...

- 人力有限，沒有專職 SRE / DevOps
- 機敏資料要求不嚴格
- 事件量不大（每月 < 數百萬筆）
- 想快速上線，不想踩坑

🏠 選 Self-Hosted 如果你是

- ...機敏資料有合規需求
- 有足夠的 SRE / DevOps 人力維運
- 事件量大，SaaS 費用難以接受
- 已有 Kubernetes / EKS 基礎設施

不確定？先從 Cloud 開始，等規模到了再評估遷移。

Q&A

有問題嗎？

歡迎針對今天分享的任何踩坑細節、架構選型、或自建 vs SaaS 的評估提出問題。



Log ≠ Error

Tracking

為什麼有了 Loki 還不夠

？兩者的設計目的不同



踩坑避坑實錄

部署、選型、SDK、版本
、資源、API



要不要自建？

Self-Hosted vs SaaS 的選
擇考量與建議